

China Software Freedom | PROJETO: 03 SISTEMA FINANCEIRO

Contexto Geral

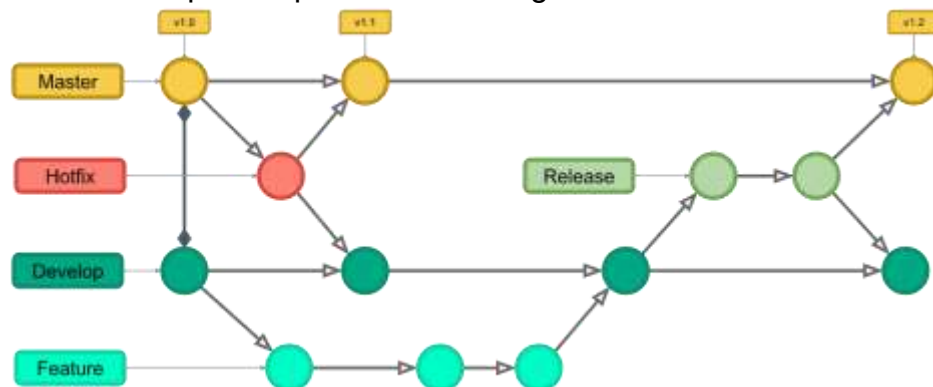
Foco: Test-Driven Development, Interfaces e Regras de Negócio em C#

Você recebeu o código inicial de um sistema ASP.NET Core MVC vinculado ao Banco Central. O sistema já possui a gestão de Agências, mas agora precisamos expandir para o setor de **Empréstimos**. Sua missão é garantir que cada centavo seja calculado corretamente através de testes automatizados.

Link github projeto inicial: <https://github.com/regymatrix/sistemafinanceiro>

ORIENTAÇÕES GIT

Use o fluxo GitFlow para implementar o código.



Especificações Gerais do Projeto

Para resolver o problema você deve criar um programa simples usando as ferramentas:

- Visual Studio
- C#
- Projeto inicial em Asp.net Core já configurado banco

Você deve fazer um FORK do projeto inicial e implementar as novas demandas dentro de sua branch

China Software Freedom | PROJETO: 03BC

Fase 01 – PREPARAÇÃO - Modelagem e expansão do domínio

Antes de calcular, precisamos de estrutura. Nesta fase, você deve preparar as entidades que suportarão o sistema de empréstimos.

Fase01A – Criar Classe Cliente e ContaBancaria

Cliente: Codigo (int), Nome (String), EstadoUF(string)

ContaBancaria: NumeroConta(String), Saldo(decimal), Cliente(Objeto), Agencia(Objeto)

FASE02 – INTERFACE EMPRESTIMO

No TDD e em arquiteturas limpas, definimos o que o sistema faz antes de decidir como ele faz.

Regra de Negócio R1: O sistema deve calcular o valor final de um empréstimo baseado em juros simples.

ValorFinal = ValorBase + (ValorBase \times Taxa)

Para organizar

1. Crie uma pasta chamada Services no seu projeto Model.
2. Crie a interface IEmprestimoService. Ela deve conter a assinatura do método: decimal CalcularJurosSimples(decimal valor, double taxa);

FASE03 – IMPLEMENTANDO TDD

Preparação do Ambiente

1. Adicione um novo projeto à sua solução do tipo Projeto de Teste do xUnit.
2. Adicione a referência do seu projeto principal (Web/Model) ao projeto de testes.

O Desafio TDD

Você não pode escrever o código do cálculo ainda! Siga estes passos:

1. Crie o Teste (**Red**): No projeto de teste, crie uma classe EmprestimoServiceTest. Escreva um método de teste que valide o seguinte cenário:
Cenário: Empréstimo de R\$ 1.000,00 com taxa de 10% (0,10).
Resultado Esperado: O valor final deve ser R\$ 1.100,00.

Execute o Teste: Ele irá falhar (ou nem compilar), pois a classe EmprestimoService ainda não foi implementada.

Implementação Mínima (Green): Agora, crie a classe EmprestimoService dentro da pasta Services (implementando a interface IEmprestimoService) e escreva o código necessário para o teste passar.

Refatoração: Melhore o código se necessário para passar

Visão das classes implementadas

```
public interface IEmprestimoService
{
    decimal CalcularJurosSimples(decimal valor, double taxa);
}
```

```
public class EmprestimoServiceTest
{
    [Fact]
    public void CalcularJurosSimples_DeveRetornarValorCorreto()
    {
        // Arrange (Preparação do cenário)
        IEmprestimoService service = new EmprestimoService();
        decimal valorEmprestimo = 1000m;
        double taxaJuros = 0.10; // 10%
        decimal resultadoEsperado = 1100m;

        // Act (Ação: Execução do método)
        decimal resultadoObtido = service.CalcularJurosSimples(valorEmprestimo, taxaJuros);

        // Assert (Verificação: O resultado é o esperado?)
        Assert.Equal(resultadoEsperado, resultadoObtido);
    }
}
```

```
public class EmprestimoService : IEmprestimoService
{
    public decimal CalcularJurosSimples(decimal valor, double taxa)
    {
        // Implementação da Regra R1
        // Valor Final = Valor + (Valor * Taxa)
        decimal valorJuros = valor * (decimal)taxa;
        decimal valorFinal = valor + valorJuros;

        return valorFinal;
    }
}
```

Fase 04 - FILOSOFANDO

01. No ciclo do TDD, por que é fundamental que o teste falhe (Red) antes de escrevermos qualquer linha de código na classe `EmprestimoService`? O que essa falha nos garante?

02. Ao criarmos a interface `IEmprestimoService` antes da implementação, estamos definindo um "contrato". Como esse contrato ajuda em um projeto onde vários desenvolvedores trabalham ao mesmo tempo?

03. Na Fase 03, você criou um projeto de teste separado. Qual a vantagem de manter os testes em um projeto distinto em vez de colocá-los dentro da mesma pasta das `Models`?

04. Imagine que a regra de cálculo mude de Juros Simples para Juros Compostos. Se tivéssemos escrito o código diretamente na `Controller` (sem usar `Services` ou `Interfaces`), qual seria a dificuldade de testar essa nova regra?

05. O que define um "bom cenário de teste"? No nosso exemplo, usamos o valor 1000 e taxa 0.10. Que outros cenários (casos de borda) você testaria para garantir que o sistema de empréstimo é à prova de erros?

06. No código da `EmprestimoService`, fizemos um cast de `double` para `decimal`. Por que, em sistemas financeiros, o tipo de dado `decimal` é preferível ao `double` ou `float`?

07. O TDD costuma resultar em um código mais "limpo". Como a necessidade de tornar um código "testável" influencia a maneira como você organiza suas classes e métodos?

08. Se um teste que antes passava (Verde) começar a falhar (Vermelho) após uma alteração no código, isso é algo ruim? Justifique sua resposta com base na segurança que os testes automatizados oferecem.

09. Na Fase 01, pedimos para a `ContaBancaria` receber um objeto `Cliente`. Como essa estrutura de "Composição de Objetos" facilita a manutenção do sistema em comparação a usar apenas variáveis soltas (como `string nomeCliente`)?

10. "Testar não é encontrar erros, é garantir a confiança". Explique essa frase relacionando-a com a responsabilidade de um desenvolvedor que trabalha em sistemas que manipulam dinheiro (como o nosso Sistema Financeiro)

FASE 05 – Sua Vez

Cálculo de Juros Compostos

Agora que você dominou o fluxo do TDD com cálculos simples, o Departamento de Risco do Banco solicitou a implementação da Regra R2. Esta regra é mais realista e foca no cálculo de empréstimos com capitalização mensal (juros sobre juros).

O Conceito Financeiro

Diferente da R1, a regra R2 utiliza a fórmula de juros compostos. Para calcular o valor final (M), utilizamos:

$$M = P \times (1 + i)^n$$

Onde:

M = Montante (Valor Final)

P = Principal (Valor do Empréstimo)

Taxa de juros por período (deve ser convertida para decimal, ex: 2% = 0.02)

Número de períodos (meses)

Exemplo Prático: Um empréstimo de R\$ 1.000,00 a uma taxa de 2% ao mês por 12 meses

$$M = 1000 \times (1 + 0.02)^{12}$$

$$M = 1000 \times (1.268241)$$

Valor Final $\approx$ R\$ 1.268,24

Dicas

01 – Atualize a interface `ImprestimoService` com o nome método e também a classe serviço correspondente

02 – crie um cenário de teste no arquivo de teste já existente

Sua implementação na classe serviço deve ser similar a:

```
public decimal CalcularJurosCompostos(decimal valor, double taxaMensal, int meses)
{
    // M = P * (1 + i)^n
    double montante = (double)valor * Math.Pow((1 + taxaMensal), meses);

    return (decimal)montante;
}
```

FASE 06A – Score

Nesta fase, vamos expandir o sistema para avaliar o risco de crédito do cliente antes de liberar o empréstimo. Para garantir que o sistema seja expansível e fácil de testar, utilizaremos o padrão de projeto Strategy

O Cenário

O sistema deve calcular uma pontuação (Score) que inicia em 1000 pontos. Cada regra de negócio pode somar ou subtrair pontos com base nos dados do Cliente e da ContaBancaria

GitHub (Crie uma Branch em seu projeto para isso bom base na Branch **feature-RegrasScore**, coloque seu nome na frente)

Estrutura Inicial

No projeto já temos a interface IRegraEscore pronta que será sua base para implementar as outras classes. Uma para cada regra, ou implementar somente 01.

```
using PrjFinanceiro.Models;

namespace PrjFinanceiro.Services
{
    public interface IRegraEscore
    {
        int CalcularPontuacao(Cliente cliente, ContaBancaria conta);
    }
}
```

Exemplo de implementação da Regra SaldoMinimo

```
using PrjFinanceiro.Models;

namespace PrjFinanceiro.Services
{
    public class RegraSaldoMinimo : IRegraEscore
    {
        public int CalcularPontuacao(Cliente cliente, ContaBancaria conta)
        {
            if (conta.Saldo < 1000)
                return -30;

            return 0;
        }
    }
}
```

Exemplo do teste

```
namespace FinanceiroTest
{
    public class RegrasScoreTest
    {
        [Fact]
        public void RegraSaldoMinimo_DeveSubtrair30Pontos_QuandoSaldoFurMenorQue1000()
        {
            // Arrange
            var regra = new RegraSaldoMinimo();
            var cliente = new Cliente { Nome = "Teste" };
            var conta = new ContaBancaria { Saldo = 500 }; // Saldo < 1000

            // Act
            int resultado = regra.CalcularPontuacao(cliente, conta);

            // Assert
            Assert.Equal(-30, resultado);
        }
    }
}
```

FASE 06B – Score Sua vez

Cada aluno (ou dupla) deve escolher ou receber uma das regras abaixo para implementar seguindo rigorosamente o fluxo TDD (Cria o teste que falha -> Implementa para passar -> Refatora)

ID	Nome da Regra	Regra de Negócio
R1	Inadimplência UF	Clientes de "RJ" ou "SP" perdem 50 pontos.
R2	Fidelidade	Clientes com UF igual à sede "SC" ganham 100 pontos.
R10	Densidade Sudeste	Clientes de "SP", "RJ", "MG" ou "ES" ganham 50 pontos.
R11	Risco Fronteira	Clientes de "RS", "PR" ou "MS" perdem 15 pontos.
R12	Incentivo Nordeste	Clientes de estados do Nordeste ganham 40 pontos.

O que você deve entregar:

Classe de Teste: Ex: `RegraInadimplenciaUFTTest.cs` com pelo menos dois cenários (um que aplica a regra e outro que não aplica).

Classe da Regra: Ex: `RegraInadimplenciaUF.cs` implementando a interface `IRegraEscore`.

GitFlow: Realize o commit na sua branch de Feature seguindo o fluxo orientado.